apps/core-backend/src/items/items.service.ts\

```
import { Injectable } from '@nestjs/common';
import { InjectRepository } from '@nestjs/typeorm';
import { Repository, In } from 'typeorm';
import { Item } from './entities/item.entity';
import { minioClient } from '../minio-client';
import { HttpService } from '@nestjs/axios';
import { firstValueFrom } from 'rxjs';
import FormData from 'form-data';

@Injectable()
export class ItemsService {
  constructor(
    @InjectRepository(Item)
    private itemsRepository: Repository<Item>,
    private readonly httpService: HttpService,
  ) { }

  async create(file: Express.Multer.File, description: string, userId: string, isLost: boolean) {
    const bucketName = 'lost-items';
    const fileName = `${Date.now()}-${file.originalname}`;

    // 1. Upload to MinIO
    await minioClient.putObject(bucketName, fileName, file.buffer);
    const imageUrl = `http://localhost:9000/${bucketName}/${fileName}`;

    // 2. Save Initial Record
    const newItem = this.itemsRepository.create({
      description,
      imageUrl,
      user: { id: userId } as any,
```

```
  isLost: isLost,
  tags: [],
  vector: []
});

const savedItem = await this.itemsRepository.save(newItem);

// 3. AI Analysis (Generate Vector)
try {
  const formData = new FormData();
  formData.append('file', file.buffer, { filename: file.originalname });
  formData.append('item_id', savedItem.id);
  formData.append('description', description || "");
  formData.append('is_lost', String(isLost));

  const response = await firstValueFrom(
    this.httpService.post('http://localhost:8000/analyze', formData, {
      headers: { ...formData.getHeaders() },
    }),
  );

  if (response.data.detected_objects) {
    savedItem.tags = response.data.detected_objects;
  }
  if (response.data.vector) {
    savedItem.vector = response.data.vector;
  }
  await this.itemsRepository.save(savedItem);
} catch (err) {
  console.error("❌ AI/Milvus Error:", err.message);
}
```

```
  // 4. SEARCH LOGIC
  // If we are reporting a LOST item, we want to find items others FOUND.
  if (isLost) {
    console.log(" 🔍 Lost Item Reported. Searching only FOUND items (Vector Search)...");

    // Pass 'false' to search for items where is_lost == false (Found items)
    // This uses the Image File for Vector Search
    const matches = await this.search(description, file, false);

    const filteredMatches = matches
      .filter(match => {
        // Rule: Exclude items uploaded by ME
        if (match.user && match.user.id === userId) {
          return false;
        }
        return true;
      })
      // Rule: Sort by Score (Highest First)
      .sort((a, b) => b.score - a.score)
      // Rule: Return Top 5
      .slice(0, 5);

    console.log(`DEBUG: Found ${matches.length} matches -> Returned top
${filteredMatches.length}`);

    return { ...savedItem, matches: filteredMatches };
  }

  return savedItem;
}
```

```
async search(queryText: string, file?: Express.Multer.File, filterIsLost?: boolean) {
  try {
    let response;

    // 🚨 FORCE IMAGE SEARCH IF FILE EXISTS
    if (file) {
      console.log("🚀 Sending Image to Python for Vector Search...");
      const formData = new FormData();
      formData.append('file', file.buffer, { filename: file.originalname });

      // Filter: If I am "Lost", I want "Found" (filterIsLost = false)
      if (filterIsLost !== undefined) {
        formData.append('filter_is_lost', String(filterIsLost));
      }

      response = await firstValueFrom(
        this.httpService.post('http://localhost:8000/search', formData, {
          headers: { ...formData.getHeaders() }
        })
      );
    }
    // Fallback to text search if no file (rare for this flow)
    else {
      const params = new URLSearchParams();
      params.append('text', queryText);
      if (filterIsLost !== undefined) params.append('filter_is_lost', String(filterIsLost));

      response = await firstValueFrom(
        this.httpService.post('http://localhost:8000/search', params, {
          headers: { 'Content-Type': 'application/x-www-form-urlencoded' }
```

```
      })
    );
  }

  const results = response.data;
  if (!results || results.length === 0) return [];

  const ids = results.map((r: any) => r.external_id);

  // Fetch details from DB
  const items = await this.itemsRepository.find({
    where: { id: In(ids) },
    relations: ['user'],
  });

  // Map scores and return
  return results.map((result: any) => {
    const item = items.find((i) => i.id === result.external_id);
    if (!item) return null;
    return { ...item, score: result.score };
  }).filter((i: any) => i !== null);

 } catch (err) {
  console.error("❌ Search Error:", err.message);
  return [];
 }
}

findAll() { return this.itemsRepository.find(); }

async findByUser(userId: string) {
```

```
    if (!userId) return [];
    return this.itemsRepository.find({
      where: { user: { id: userId } },
      order: { createdAt: 'DESC' },
    });
  }

  async nuke() {
    console.log("☢️ NUKING DATABASE ☢️");

    // 1. Clear Postgres
    await this.itemsRepository.clear(); // Truncates the table

    // 2. Clear Milvus
    try {
      await firstValueFrom(this.httpService.delete('http://localhost:8000/reset'));
      console.log("✅ Milvus reset successfully");
    } catch (e) {
      console.error("❌ Failed to reset Milvus:", e.message);
    }

    return { status: 'Database Nuked 🤯' };
  }
}
```

apps/core-backend/src/auth/auth.service.ts

```
import { Injectable, UnauthorizedException, BadRequestException } from '@nestjs/common';
import { UsersService } from '../users/users.service';
import { JwtService } from '@nestjs/jwt';
import * as bcrypt from 'bcrypt';
import * as crypto from 'crypto';
```

```
import { EmailService } from './email.service';

@Injectable()
export class AuthService {
  constructor(
    private usersService: UsersService,
    private jwtService: JwtService,
    private emailService: EmailService,
  ) { }

  async validateUser(email: string, pass: string): Promise<any> {
    const user = await this.usersService.findOneByEmail(email);
    if (user && await bcrypt.compare(pass, user.passwordHash)) {
      const { passwordHash, ...result } = user;
      return result;
    }
    return null;
  }

  async login(user: any) {
    const payload = { sub: user.id, email: user.email, name: user.fullName };
    return {
      access_token: await this.jwtService.signAsync(payload),
    };
  }

  async forgotPassword(email: string) {
    const user = await this.usersService.findOneByEmail(email);
    if (!user) return { message: 'If email exists, reset link sent.' };

    const token = crypto.randomBytes(32).toString('hex');
```

```
    const expires = new Date();
    expires.setHours(expires.getHours() + 1);

    await this.usersService.setResetToken(user.id, token, expires);
    await this.emailService.sendResetEmail(user.email, token);

    return { message: 'Reset email sent.' };
  }

  async resetPassword(token: string, newPass: string) {
    const user = await this.usersService.findByResetToken(token);

    if (!user || !user.resetPasswordExpires || user.resetPasswordExpires < new Date()) {
      throw new BadRequestException('Invalid or expired token');
    }

    const hashedPassword = await bcrypt.hash(newPass, 10);
    await this.usersService.updatePasswordAndClearToken(user.id, hashedPassword);

    return { message: 'Password reset successful. Please login.' };
  }
}
```

apps/core-backend/src/items/items.controller.ts

```
import { Controller, Post, UseInterceptors, UploadedFile, Body, Get, Req, UseGuards, Delete } from '@nestjs/common';
import { FileInterceptor } from '@nestjs/platform-express';
import { ItemsService } from './items.service';
import { DebugAuthGuard } from '../auth/debug-auth.guard'; // Keeping your verified guard

@Controller('items')
```

```
export class ItemsController {
  constructor(private readonly itemsService: ItemsService) { }

  @Post()
  @UseGuards(DebugAuthGuard) // 🔒 Protects this route
  @UseInterceptors(FileInterceptor('file'))
  async create(
    @UploadedFile() file: Express.Multer.File,
    @Body('description') description: string,
    @Body('type') type: string, // 'lost' or 'found'
    @Req() req: any, // Access the logged-in user
  ) {
    // ✅ Extract Real User ID from the Token (from DebugAuthGuard/JwtStrategy)
    // JwtStrategy returns { id, email, name }
    const userId = req.user.id;
    console.log(`📝 Uploading Item for User ID: ${userId}`);

    // Convert string to boolean
    const isLost = type === 'lost';
    console.log(`📝 Item Type: '${type}', isLost: ${isLost}`);

    return this.itemsService.create(file, description, userId, isLost);
  }

  @Post('search')
  async search(@Body('text') text: string) {
    return this.itemsService.search(text);
  }

  @Get()
  findAll() {
```

```
    return this.itemsService.findAll();
  }

  @UseGuards(DebugAuthGuard) // Protect this route
  @Get('mine')
  async getMyItems(@Req() req) {
    // req.user is populated by the Guard (from the Token via JwtStrategy)
    // JwtStrategy returns { id, email, name }
    const userId = req.user.id;
    return this.itemsService.findByUser(userId);
  }



  @Delete('nuke')
  async nukeDatabase() {
    console.warn("☢️ NUKING DATABASE: Deleting all items...");
    return this.itemsService.nuke();
  }
}
```

apps/core-backend/src/items/entities/item.entity.ts

```
import { Entity, PrimaryGeneratedColumn, Column, ManyToOne, CreateDateColumn } from
'typeorm';
import { User } from '../../users/entities/user.entity';

@Entity()
export class Item {
  @PrimaryGeneratedColumn('uuid')
  id: string;

  @Column()
```

```
  description: string;

  @Column()
  imageUrl: string; // URL to MinIO

  @Column("simple-array", { nullable: true })
  tags: string[]; // ["backpack", "red"] from YOLO

  @Column("float", { array: true, nullable: true })
  vector: number[]; // [0.12, -0.5...] from CLIP

  @Column({ default: false })
  isLost: boolean; // true = lost, false = found

  @ManyToOne(() => User, (user) => user.id)
  user: User; // Who posted this?

  @CreateDateColumn()
  createdAt: Date;
}
```